In [337…

```
%pip install moabb mne
```

```
Note: you may need to restart the kernel to use updated packages.
```

In [338…
```python
from moabb.datasets import BNCI2014_009
from moabb.paradigms import P300
import numpy as np
import pandas as pd

dataset = BNCI2014_009()
paradigm = P300()

X_raw, y_raw, metadata = paradigm.get_data(
    dataset=dataset,
    subjects=[1],
    return_epochs=False
)

print(f"X_raw форма: {X_raw.shape}")
print(f"y_raw форма: {y_raw.shape}")

y = np.array([1 if label == 'Target' else 0 for label in y_raw])

print(f"\nПосле преобразования меток:")
print(f"Target (1): {np.sum(y == 1)}")
print(f"NonTarget (0): {np.sum(y == 0)}")

X_epochs = X_raw
```

```
X_raw форма: (1728, 16, 206)
y_raw форма: (1728,)

После преобразования меток:
Target (1): 288
NonTarget (0): 1440
```

In [339…
```python
print(f"Форма X_epochs: {X_epochs.shape}")

p300_channel_indices = [1, 2, 3, 6]
print(f"\nИспользуем каналы с индексами: {p300_channel_indices}")

X_avg = np.mean(X_epochs[:, p300_channel_indices, :], axis=1)

n_blocks = 32
n_times = X_avg.shape[1]
block_size = n_times // n_blocks

for i in range(n_blocks):
    start = i * block_size
    end = start + block_size
    block_mean = np.mean(X_avg[:, start:end], axis=1)
    X_features.append(block_mean)

X = np.array(X_features).T
```

```
mean = np.mean(X, axis=0)
std = np.std(X, axis=0)
std[std == 0] = 1
X = (X - mean) / std
print(f"После нормализации: min={X.min():.2f}, max={X.max():.2f}")

print(f"\nИтог:")
print(f" Target (1): {np.sum(y == 1)}")
print(f" NonTarget (0): {np.sum(y == 0)}")
```

```
Форма X_epochs: (1728, 16, 206)

Используем каналы с индексами: [1, 2, 3, 6]
После нормализации: min=-4.06, max=5.17

Итог:
 Target (1): 288
 NonTarget (0): 1440
```

In [ ]:

```
class SVM:
    def __init__(self, learning_rate=0.001, C=10.0, n_iterations=5000):
        self.lr = learning_rate
        self.C = C
        self.n_iterations = n_iterations
        self.w = None
        self.b = None
        self.losses = []

    def fit(self, X, y, class_weight='balanced'):

        n_samples, n_features = X.shape

        if class_weight == 'balanced':
            n_target = np.sum(y == 1)
            n_nontarget = np.sum(y == 0)
            weight_1 = n_nontarget / n_target
            weight_0 = 1.0
            print(f"Балансировка, вес для Target (1) = {weight_1:.2f}")
        elif isinstance(class_weight, dict):
            weight_0 = class_weight.get(0, 1.0)
            weight_1 = class_weight.get(1, 1.0)
        else:
            weight_0 = 1.0
            weight_1 = 1.0

        y_svm = np.where(y == 0, -1, 1)

        self.w = np.random.randn(n_features) * 0.01
        self.b = 0.0


        for iteration in range(self.n_iterations):
            dw = np.zeros(n_features)
```

```
            db = 0.0

            for i in range(n_samples):
                condition = y_svm[i] * (np.dot(X[i], self.w) + self.b)
                if condition < 1:

                    weight = weight_1 if y[i] == 1 else weight_0
                    dw += -self.C * y_svm[i] * X[i] * weight
                    db += -self.C * y_svm[i] * weight

            dw += self.w
            self.w -= self.lr * dw
            self.b -= self.lr * db

            if iteration % 100 == 0:

                loss = 0.5 * np.dot(self.w, self.w)
                for i in range(n_samples):
                    weight = weight_1 if y[i] == 1 else weight_0
                    margin = y_svm[i] * (np.dot(X[i], self.w) + self.b)
                    loss += self.C * weight * max(0, 1 - margin)
                self.losses.append(loss)


        print("Обучение завершено")

    def predict(self, X, return_scores=False):
        scores = np.dot(X, self.w) + self.b
        if return_scores:
            return scores
        return np.where(scores >= 0, 1, 0)
```

In [341…

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Разделение данных:")
print(f"  Обучающая выборка: {X_train.shape[0]} примеров")
print(f"  Тестовая выборка: {X_test.shape[0]} примеров")
print(f"  Train Target: {np.sum(y_train==1)}, NonTarget: {np.sum(y_train==0)}"
print(f"  Test Target: {np.sum(y_test==1)}, NonTarget: {np.sum(y_test==0)}")
```

```
Разделение данных:
  Обучающая выборка: 1382 примеров
  Тестовая выборка: 346 примеров
  Train Target: 230, NonTarget: 1152
  Test Target: 58, NonTarget: 288
```

In [372…

```
clf = SVM(learning_rate=0.01, C=100.0, n_iterations=5000)
clf.fit(X_train, y_train)

y_scores = clf.predict(X_test, return_scores=True)
```

```
THRESHOLD = 1.0
y_pred = np.where(y_scores >= THRESHOLD, 1, 0)

print(f"Порог: {THRESHOLD}")
print(f"Предсказано Target (1): {np.sum(y_pred == 1)}")
print(f"Первые 10 предсказаний: {y_pred[:10]}")
print(f"Первые 10 реальных меток: {y_test[:10]}")
```

```
Балансировка, вес для Target (1) = 5.01
Обучение завершено
Порог: 1.0
Предсказано Target (1): 145
Первые 10 предсказаний: [1 1 0 0 1 0 0 0 0 0]
Первые 10 реальных меток: [0 0 0 0 1 1 0 0 0 0]
```

In [373…

```
def calculate_metrics(y_true, y_pred):

    tp = np.sum((y_true == 1) & (y_pred == 1))
    tn = np.sum((y_true == 0) & (y_pred == 0))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))


    accuracy = (tp + tn) / (tp + tn + fp + fn)
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0
    f1 = 2 * (precision * recall) / (precision + recall) if (precision + recal

    print(f"Матрица ошибок:")
    print(f"            Предсказано 0  Предсказано 1")
    print(f"Было 0:          {tn:4d}          {fp:4d}")
    print(f"Было 1:          {fn:4d}          {tp:4d}")
    print(f"\nМетрики:")
    print(f"  Accuracy:  {accuracy:.4f} ")
    print(f"  Precision: {precision:.4f} ")
    print(f"  Recall:    {recall:.4f} ")
    print(f"  F1-Score:  {f1:.4f}  )")

    return {'accuracy': accuracy, 'precision': precision, 'recall': recall, 'f
            'tp': tp, 'tn': tn, 'fp': fp, 'fn': fn}

metrics = calculate_metrics(y_test, y_pred)
```

```
Матрица ошибок:
            Предсказано 0  Предсказано 1
Было 0:           181           107
Было 1:            20            38

Метрики:
  Accuracy:  0.6329
  Precision: 0.2621
  Recall:    0.6552
  F1-Score:  0.3744  )
```

In [374…

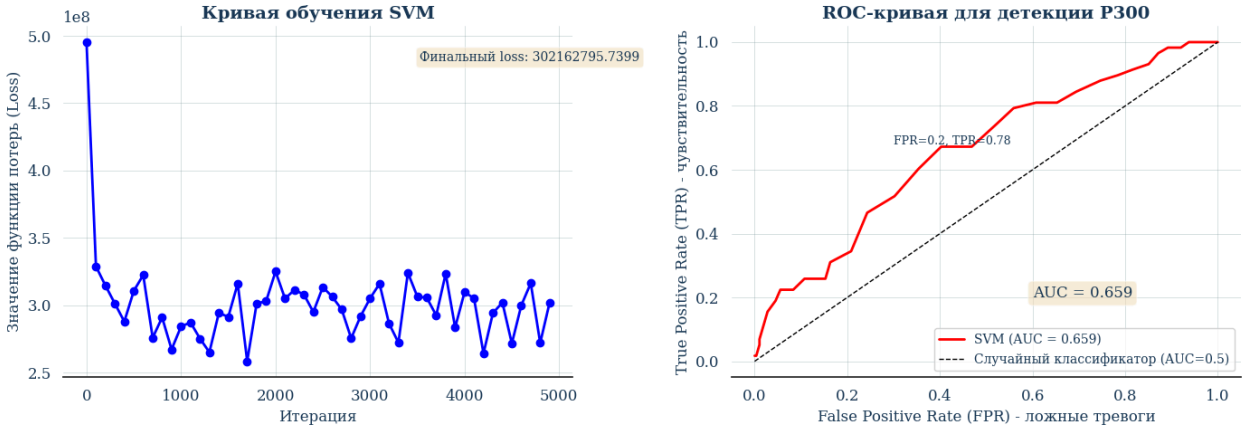
```python
def calculate_roc(y_true, y_scores, n_points=50):

    thresholds = np.linspace(np.min(y_scores), np.max(y_scores), n_points)
    tpr_list = []
    fpr_list = []

    for thresh in thresholds:
        y_pred_thresh = np.where(y_scores >= thresh, 1, 0)

        tp = np.sum((y_true == 1) & (y_pred_thresh == 1))
        fp = np.sum((y_true == 0) & (y_pred_thresh == 1))
        fn = np.sum((y_true == 1) & (y_pred_thresh == 0))
        tn = np.sum((y_true == 0) & (y_pred_thresh == 0))

        tpr = tp / (tp + fn) if (tp + fn) > 0 else 0
        fpr = fp / (fp + tn) if (fp + tn) > 0 else 0

        tpr_list.append(tpr)
        fpr_list.append(fpr)

    return fpr_list, tpr_list, thresholds


fpr, tpr, thresholds = calculate_roc(y_test, y_scores)
from sklearn.metrics import auc
auc_value = auc(fpr, tpr)

print(f"AUC = {auc_value:.4f}")
```

```
AUC = 0.6589
```

In [375…

```python
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(range(0, clf.n_iterations, 100), clf.losses, 'b-o', linewidth=2,
axes[0].set_title('Кривая обучения SVM', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Итерация', fontsize=12)
axes[0].set_ylabel('Значение функции потерь (Loss)', fontsize=12)
axes[0].grid(True, alpha=0.3)

axes[0].text(0.7, 0.9, f'Финальный loss: {clf.losses[-1]:.4f}',
             transform=axes[0].transAxes, fontsize=10,
             bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

axes[1].plot(fpr, tpr, 'r-', linewidth=2, label=f'SVM (AUC = {auc_value:.3f})'
axes[1].plot([0, 1], [0, 1], 'k--', linewidth=1, label='Случайный классификатс
axes[1].set_xlabel('False Positive Rate (FPR) - ложные тревоги', fontsize=12)
axes[1].set_ylabel('True Positive Rate (TPR) - чувствительность', fontsize=12)
axes[1].set_title('ROC-кривая для детекции P300', fontsize=14, fontweight='bol
axes[1].legend(loc='lower right', fontsize=10)
axes[1].grid(True, alpha=0.3)

axes[1].text(0.6, 0.2, f'AUC = {auc_value:.3f}', fontsize=12,
```

```
            bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

optimal_fpr = 0.2
optimal_tpr = 0.78

axes[1].annotate(f'FPR={optimal_fpr}, TPR={optimal_tpr}',
                 xy=(optimal_fpr, optimal_tpr), xytext=(optimal_fpr+0.1, optim
                 fontsize=9)

plt.tight_layout()
plt.show()
```



In [376…

```
print(f"    Accuracy: {metrics['accuracy']:.2%} ")
print(f"    Precision: {metrics['precision']:.2%}")
print(f"    Recall: {metrics['recall']:.2%} ")

print(f"\n3. AUC = {auc_value:.3f} ")
```

```
   Accuracy: 63.29%
   Precision: 26.21%
   Recall: 65.52%

3. AUC = 0.659
```